

UNIVERSIDADE FEDERAL DE RORAIMA

Curso de Ciência da Computação — Arquitetura e Organização de Computadores

Relatório Técnico — Laboratório de Circuitos

Projeto Integrador: Subsistema de Memória e Unidade de Controle

Campo	Preenchimento
Professor	Prof. Dr. Herbert Oliveira Rocha
Semestre	2026.2
Integrante 1	Alonso Fernandes Cavalcante - Matrícula: 2024005534
Integrante 2	Gwylherme Mota Rodrigues - Matrícula: 2024002200
Repositório	https://github.com/gwylhermm/AOC_AlonsoGwylhermm_UFRR_LabCircuitos_2026.git

1. Introdução

Este relatório documenta a Parte I do Projeto Integrador: um módulo de memória de 64 bytes com decodificação de endereços e uma cache de mapeamento direto de 4 linhas, simulados no Logisim-Evolution. A avaliação recai sobre o subsistema montado e testado, e não sobre componentes isolados.

Objetivos: (i) explicar como vários dispositivos compartilham um barramento e qual o papel do decodificador de endereços; (ii) montar o módulo de memória com quatro regiões selecionadas por um decodificador $2 \rightarrow 4$; (iii) decompor endereços de 6 bits em rótulo, índice e deslocamento e classificar cada acesso de um traço; (iv) calcular a taxa de acertos e o tempo médio de acesso (AMAT).

Cada componente é apresentado com a descrição dos pinos, a imagem do circuito e os testes realizados. Os arquivos da Parte I são **parte1_memoria.circ**, **parte1_cache.circ** e **parte1_planilha.xlsx**.

2. Parte I — Subsistema de memória

2.1 Mapa de memória e seleção de região

O barramento de endereços tem 6 bits (A5..A0), totalizando 64 posições, e o de dados tem 8 bits. Os bits A5 e A4 escolhem a região por meio do decodificador; os bits A3..A0 endereçam uma das 16 posições internas. A Tabela 1 mostra o mapa e as equações de seleção.

Tabela 1 — Mapa de memória e equações de seleção

Faixa	Dispositivo	Tamanho	Sinal de seleção
0x00 – 0x0F	ROM de programa	16 bytes	$CS_ROM = A5' \cdot A4'$
0x10 – 0x1F	RAM de dados	16 bytes	$CS_RAM = A5' \cdot A4$
0x20 – 0x2F	Banco de registradores	16 bytes	$CS_REG = A5 \cdot A4'$
0x30 – 0x3F	Região de E/S	16 bytes	$CS_IO = A5 \cdot A4$

A Tabela 2 é a tabela verdade de A5 e A4. Em cada combinação, exatamente um dos quatro sinais CS fica ativo.

Tabela 2 — Tabela verdade dos sinais de seleção

A5	A4	CS_ROM	CS_RAM	CS_REG	CS_IO
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

A saída de dados é escolhida por um multiplexador de 4 entradas comandado pelos mesmos bits A5 e A4 (Figura 3). A região de E/S não tem dispositivo nesta versão, e a entrada E3 do multiplexador está fixa em 0x00.

2.2 Divisão do endereço para a cache

Com bloco de 2 bytes e 4 linhas, os campos do endereço de 6 bits são:

deslocamento = $\log_2(\text{tamanho do bloco}) = \log_2(2) = 1 \text{ bit} \rightarrow A0$

índice = $\log_2(\text{número de linhas}) = \log_2(4) = 2 \text{ bits} \rightarrow A2 A1$

rótulo = $6 - 2 - 1 = 3 \text{ bits} \rightarrow A5 A4 A3$

Exemplo: 0x22 = 100010. Da direita para a esquerda, o deslocamento é 0, o índice é 01 e o rótulo é 100. Esse endereço só pode ocupar a linha 1, onde será registrado o rótulo 100.

2.3 Componentes do módulo de memória

O circuito principal *main* integra os dispositivos (Figura 1). Os componentes são descritos a seguir.

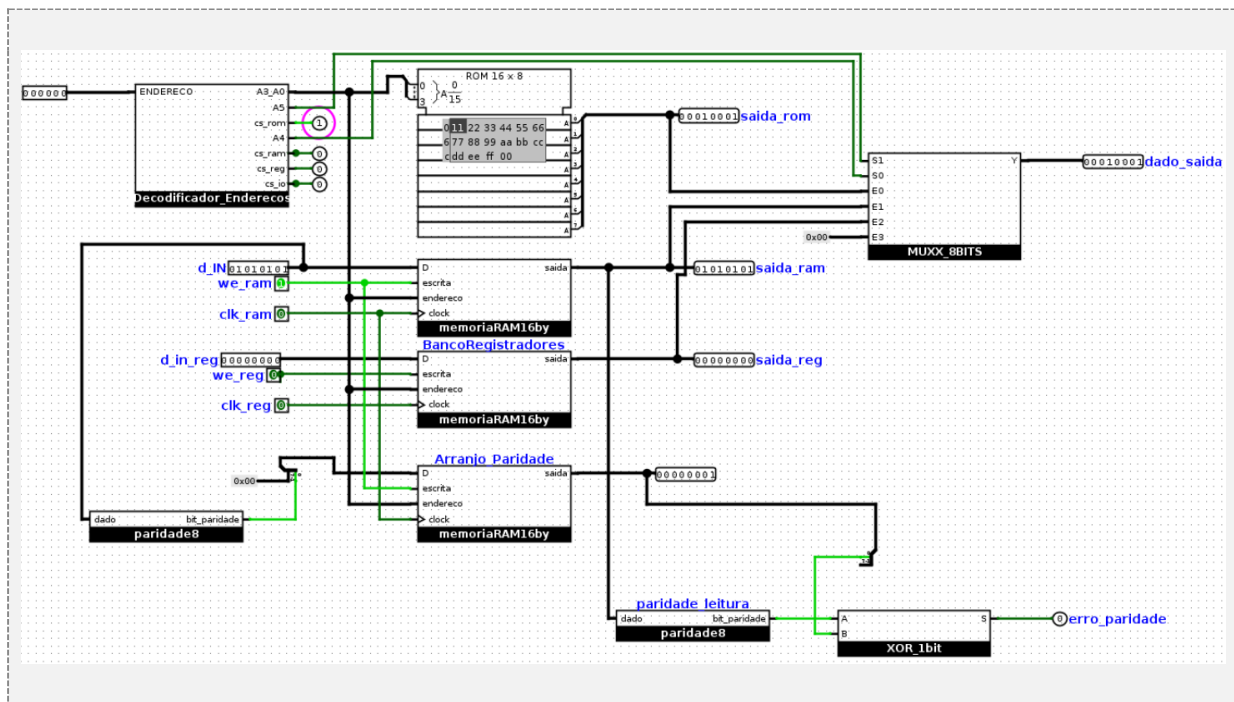


Figura 1 — Circuito principal do módulo de memória (main)

2.3.1 Decodificador de endereços

Duas portas NOT e quatro portas AND implementam diretamente as equações da Tabela 1 (Figura 2). Os bits A5 e A4 são separados do endereço por um splitter.

Tabela 3 — Pinos do Decodificador_Enderecos

Pino	Bits	Dir.	Função
ENDEREÇO	6	entrada	Endereço completo (A5..A0)
A5, A4	1+1	saída	Bits que escolhem a região (para o MUX de saída)
A3_A0	4	saída	Endereço interno na região
cs_rom, cs_ram, cs_reg, cs_io	1 cada	saída	Seleção de cada região (um único ativo)

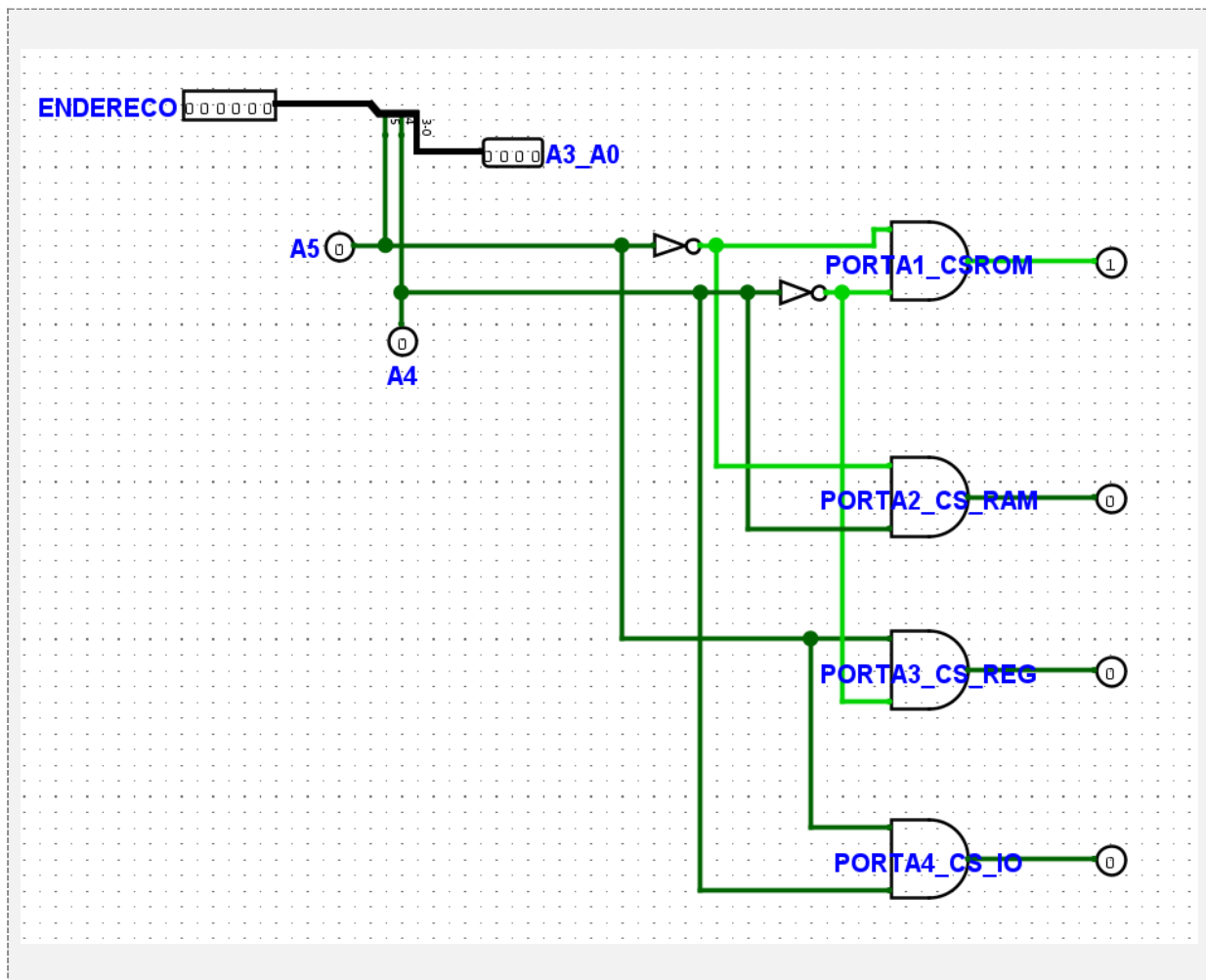


Figura 2 — Decodificador de endereços

2.3.2 Multiplexador de saída (MUXX_8BITS)

Seleciona entre as saídas da ROM (E0), da RAM (E1), do banco de registradores (E2) e da região de E/S (E3, fixa em 0x00), comandado por S1 = A5 e S0 = A4. É formado por multiplexadores de 1 bit, um por bit de dado (Figura 3).

Tabela 4 — Pinos do MUXX_8BITS

Pino	Bits	Dir.	Função
E0, E1, E2, E3	8	entrada	Dados dos quatro dispositivos
S1, S0	1+1	entrada	Seleção (A5, A4)
Y	8	saída	Dado do dispositivo escolhido

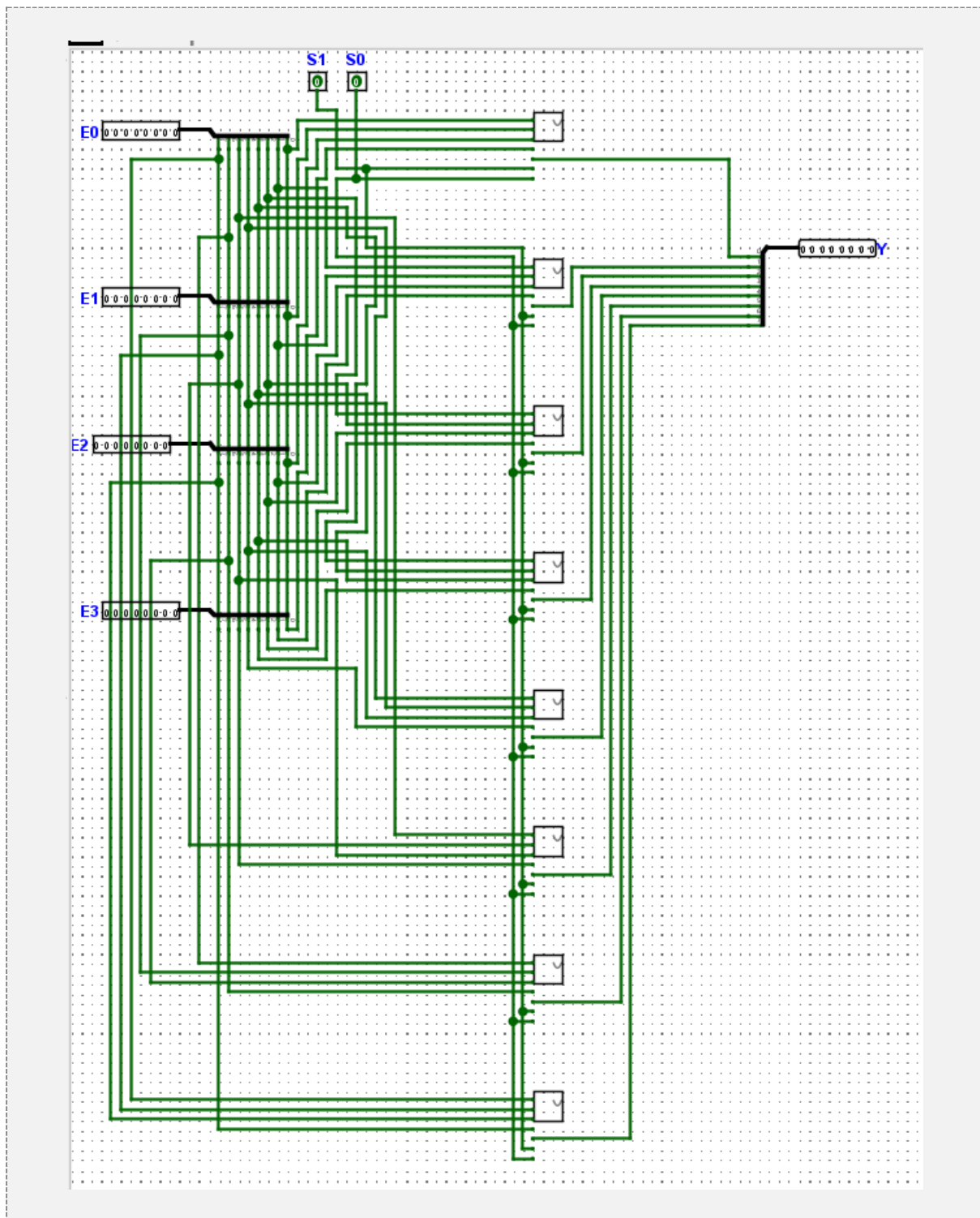


Figura 3 — Multiplexador de saída de 8 bits

2.3.3 Banco de registradores de 8 bits (região 0x20–0x2F)

O banco ocupa a região 0x20–0x2F: $CS_REG = A5 \cdot A4'$ seleciona o banco, e A3..A0 escolhem um dos 16 registradores. Ele foi construído a partir de flip-flops D, sem usar a RAM ou a memória da biblioteca (Figura 4 e Figura 5):

- **registrador8bits**: oito flip-flops D; um multiplexador 2:1 por bit realimenta a saída q quando $we = 0$ e carrega d quando $we = 1$, de modo que o valor só muda na borda do clock com $we = 1$.

- **memoriaRAM16by** (instância *BancoRegistadores*): 16 registradores, um decodificador 4→16 sobre o endereço, 16 portas AND (saída do decodificador AND escrita) que geram o we de cada registrador, e um multiplexador 16:1 que leva ao pino saída o registrador endereçado.

A mesma implementação de 16 × 8 bits é reutilizada para a RAM de dados (0x10–0x1F) e para o arranjo de paridade. O banco tem uma porta de leitura; a leitura dos dois operandos do caminho de dados (rs e rt) será tratada na Parte II.

Tabela 5 — Pinos do memoriaRAM16by (banco de registradores)

Pino	Bits	Dir.	Função
D	8	entrada	Dado a gravar
escrita	1	entrada	Habilita a gravação
endereço	4	entrada	Registrador escolhido (A3..A0)
clock	1	entrada	Relógio (grava na borda de subida)
saída	8	saída	Conteúdo do registrador endereçado

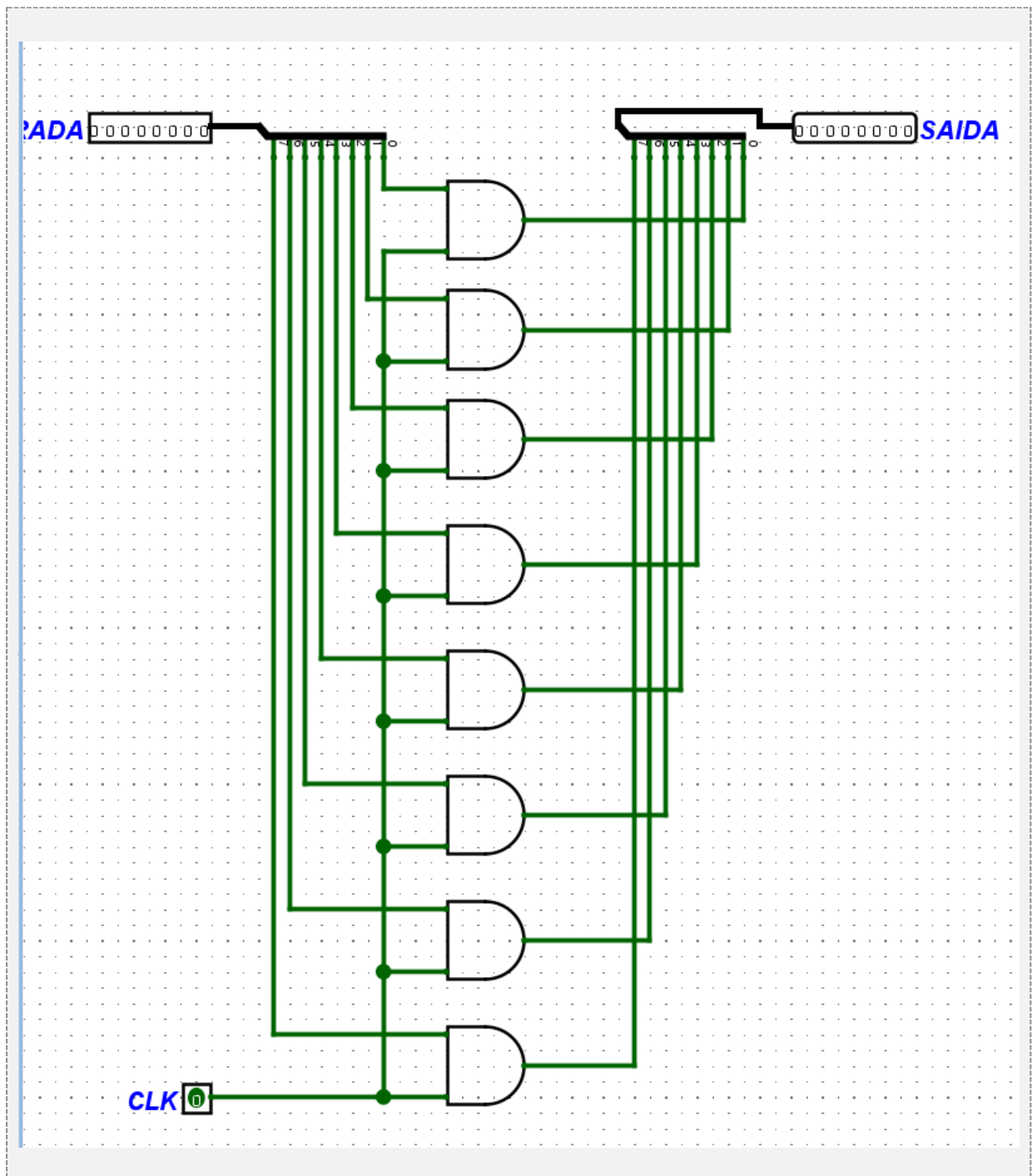


Figura 4 — registrador8bits: oito flip-flops D com multiplexador de carga

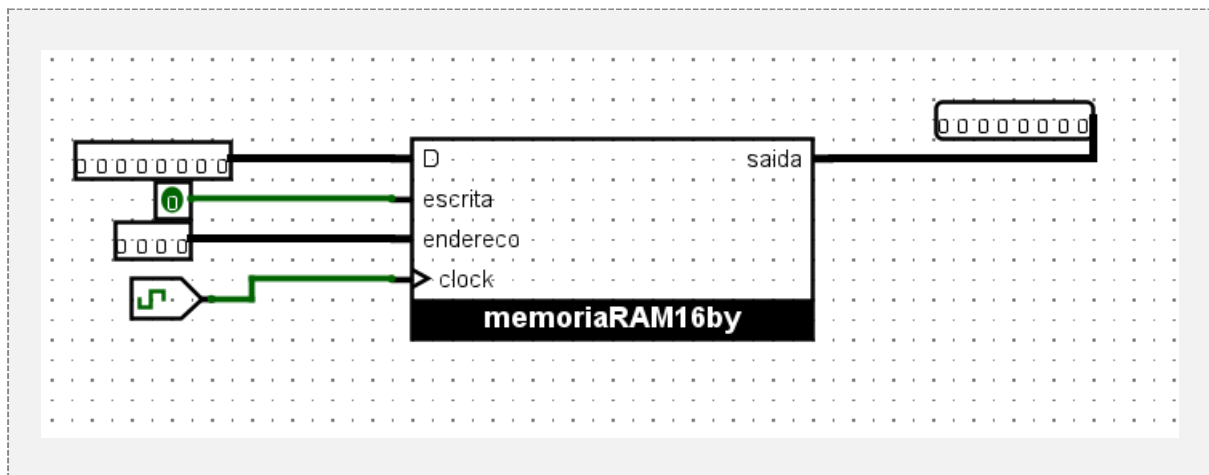


Figura 5 — memoriaRAM16by: 16 registradores, decodificador, portas AND e multiplexador 16:1

Testes do banco (Tabela 6). O endereço 0x23 é 100011: A5A4 = 10 seleciona o banco e A3..A0 = 0011 escolhe o registrador 3.

Tabela 6 — Testes do banco de registradores

ID	Entradas	Saída observada	Evidência
REG-1	endereço = 100011; d_in_reg = 0xA5; we_reg = 1; pulso em clk_reg; we_reg = 0	cs_reg = 1; saída_reg = 0xA5; dado_saída = 0xA5	REG-1
REG-2	endereço = 100100 (0x24)	saída_reg = 0x00; dado_saída = 0x00	REG-2
REG-3	endereço = 100011; d_in_reg = 0xFF; we_reg = 0; pulso em clk_reg	saída_reg continua 0xA5 (sem gravação)	REG-3

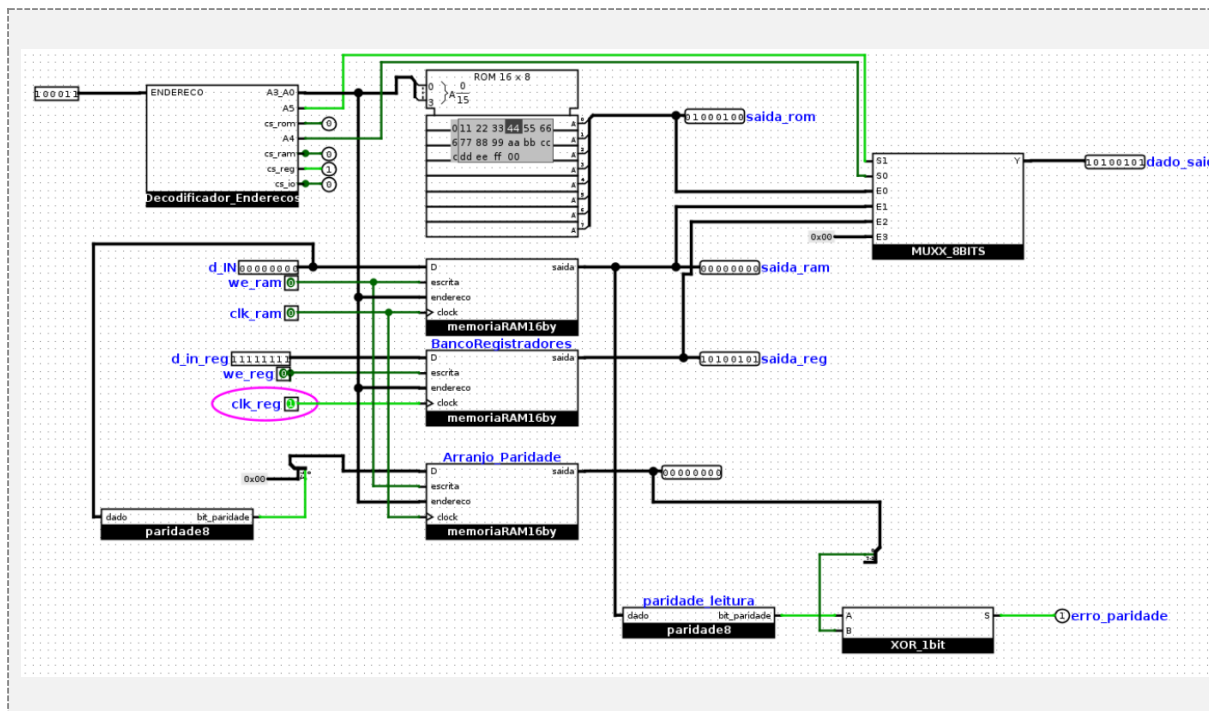




Figura 6 — Evidências do teste do banco de registradores

2.3.4 RAM de dados e bit de paridade ímpar

A RAM de dados (0x10–0x1F) usa o mesmo circuito memóriaRAM16by. Cada palavra gravada gera um bit de paridade ímpar pelo circuito **paridade8**, guardado no *Arranjo_Paridade*. Na leitura, a paridade é recalculada pelo *paridade_leitura* (outra instância de *paridade8*) e comparada com a guardada por um *XOR_1bit*; se divergirem, *erro_paridade* = 1.

O *paridade8* divide o byte em dois grupos de quatro bits (subcircuitos *paridade4*, rotulados *P_baixo* e *P_alto*) e combina os resultados com portas AND, OR e NOT, invertendo a saída. O bit gerado torna ímpar o total de bits 1 do dado mais a paridade (por exemplo, 0x55 tem quatro bits 1, e a paridade guardada é 1) (Figura 7).

Tabela 7 — Pinos do *paridade8*

Pino	Bits	Dir.	Função
dado	8	entrada	Byte cuja paridade é calculada
bit_paridade	1	saída	Bit que torna ímpar o total de 1s

2.3.5 ROM de programa

Região 0x00–0x0F: memória ROM de 16 × 8 bits da biblioteca do Logisim, carregada com os valores de teste 0x11, 0x22, ..., 0xFF, 0x00 nos endereços 0 a F. Na Parte II ela guarda o programa.

2.4 A cache

Parâmetros fixos (Tabela 8): mapeamento direto, 4 linhas, bloco de 2 bytes, capacidade de dados de 8 bytes, 1 bit de validade e 3 bits de rótulo por linha, somente leitura, tempo de acerto de 1 ciclo e penalidade de falta de 8 ciclos.

Tabela 8 — Parâmetros da cache

Parâmetro	Valor
Organização	Mapeamento direto
Número de linhas	4
Tamanho do bloco	2 bytes
Capacidade de dados	8 bytes
Metadados por linha	1 bit de validade + 3 bits de rótulo
Operações atendidas	Somente leitura
Tempo de acerto	1 ciclo
Penalidade de falta	8 ciclos

2.4.1 XOR construída com AND, NOT e OR

O Componente 03 foi implementado como $A \cdot B' + A' \cdot B$, usando duas portas NOT, duas AND e uma OR, sem a porta XOR da biblioteca (Figura 8).

Tabela 9 — Pinos do XOR_1bit

Pino	Bits	Dir.	Função
A, B	1+1	entrada	Operandos
S	1	saída	A XOR B

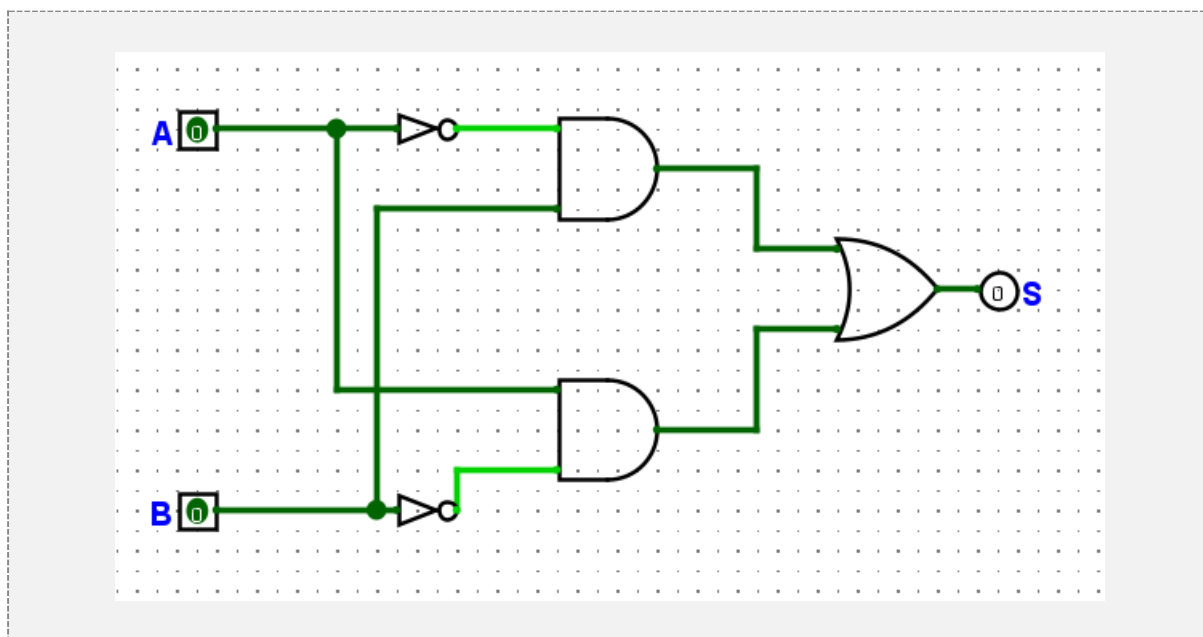


Figura 8 — XOR de 1 bit a partir de AND, NOT e OR

2.4.2 Linha da cache e conjunto de linhas

Cada **LinhaCache** contém um registrador de 3 bits para o rótulo e um flip-flop JK para o bit de validade ($J = \text{escrever}$, $K = 0$, reset = zerar), o que permite invalidar a cache. O **cachelinhas** reúne quatro linhas; um decodificador $2 \rightarrow 4$ sobre o índice e quatro portas AND com escrever habilitam apenas a linha indexada, e dois multiplexadores $4:1$ levam à saída o rótulo e o bit de validade da linha escolhida (Figura 9 e Figura 10).

Tabela 10 — Pinos do LinhaCache

Pino	Bits	Dir.	Função
rotulo_in	3	entrada	Rótulo a gravar
escrever	1	entrada	Habilita a gravação
clk	1	entrada	Relógio
zerar	1	entrada	Invalida a linha (validade = 0)
rotulo_out	3	saída	Rótulo guardado
valido	1	saída	Bit de validade

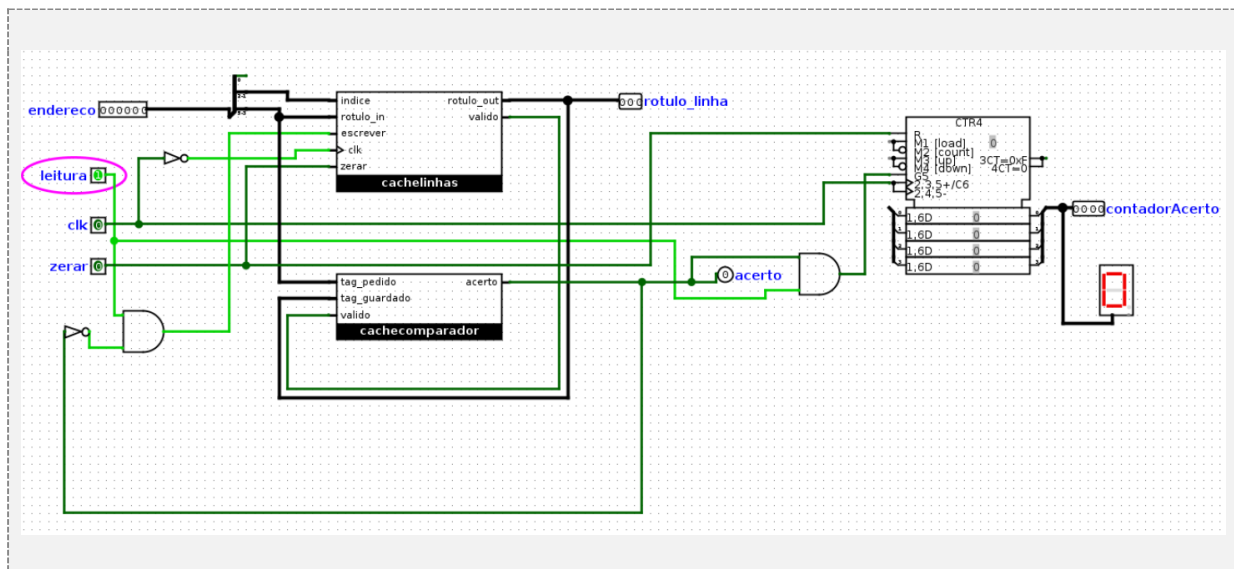


Figura 9 — LinhaCache: rótulo e bit de validade

Tabela 11 — Pinos do cachelinhas

Pino	Bits	Dir.	Função
indice	2	entrada	Linha acessada (A2 A1)
rotulo_in	3	entrada	Rótulo do endereço pedido
escrever, clk, zerar	1 cada	entrada	Controle
rotulo_out	3	saída	Rótulo guardado na linha indexada
valido	1	saída	Validade da linha indexada

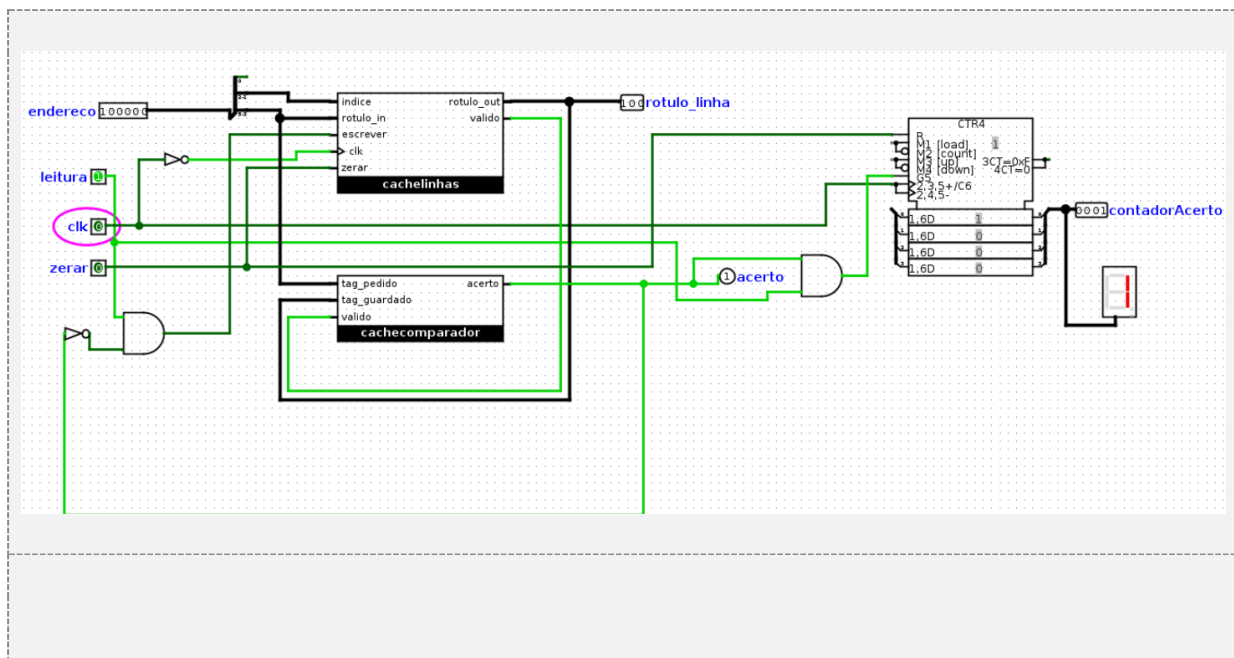


Figura 10 — cachelinhas: quatro linhas, decodificador e multiplexadores

2.4.3 Comparador e acerto

O cachecomparador implementa as expressões da Seção 5.3.2 do enunciado: três XOR_1bit comparam bit a bit o rótulo pedido (T) e o guardado (S); uma porta OR reúne as diferenças; um NOT produz iguais; e um AND com o bit de validade produz ACERTO (Figura 11).

Tabela 12 — Pinos do cachecomparador

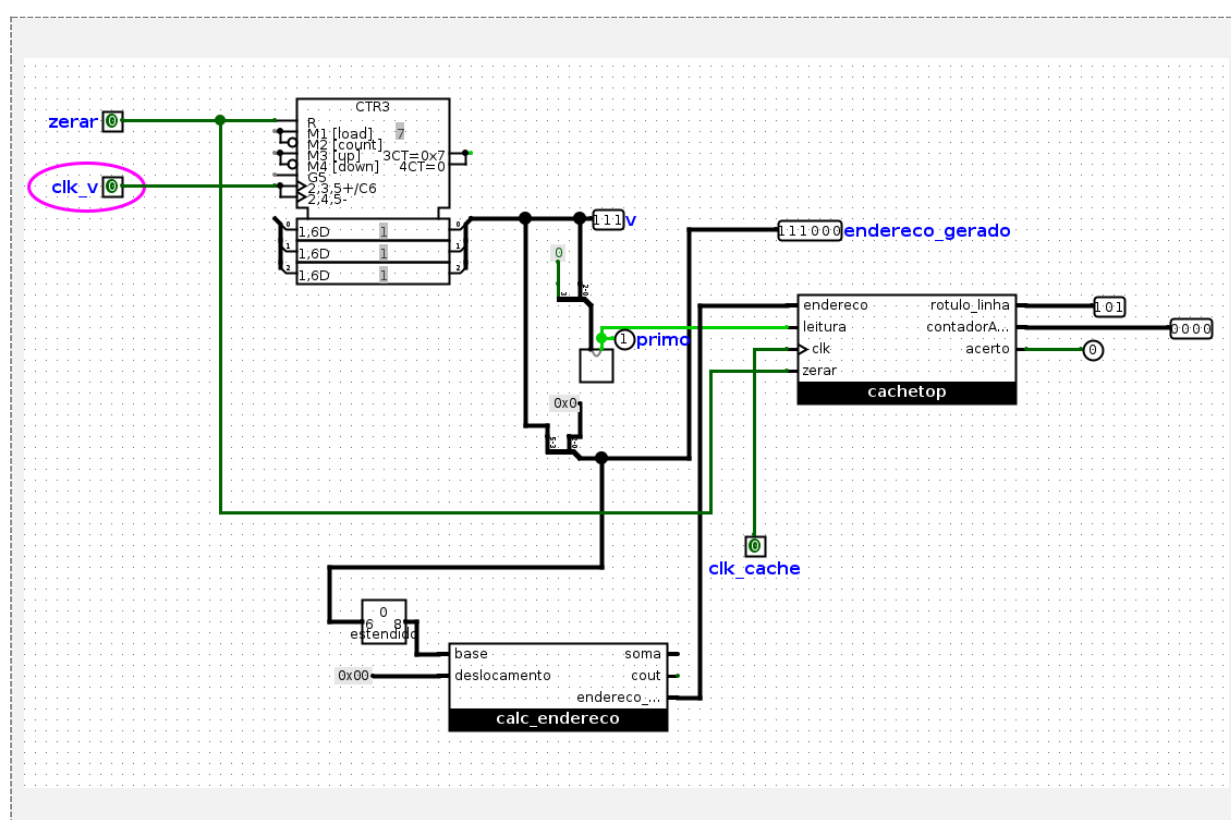


Figura 11 — Comparador de rótulos

O cachetop separa o endereço nos campos índice, rótulo e deslocamento, liga o cachelinhas ao comparador e gera escrever = leitura AND NOT acerto, de modo que a linha só é gravada em uma falta. A gravação ocorre quando o clk desce de 1 para 0 (há um inversor na entrada de relógio do cachelinhas). Um contador de 4 bits conta os acertos e alimenta um display de sete segmentos (Figura 12).

Tabela 13 — Pinos do cachetop

Pino	Bits	Dir.	Função
endereço	6	entrada	Endereço pedido (A5..A0)
leitura	1	entrada	Habilita o acesso de leitura
clk	1	entrada	Relógio
zerar	1	entrada	Invalida a cache e zera o contador
rotulo_linha	3	saída	Rótulo da linha indexada
contadorAcerto	4	saída	Número de acertos
acerto	1	saída	1 = acerto

2.5 Testes obrigatórios da Parte I

A Tabela 14 resume os testes T-01 a T-05 da tabela de testes obrigatórios do enunciado. Em cada caso foram registrados os pinos de entrada acionados, os valores nos pinos de saída e a captura de tela do simulador (Figura 13 a Figura 17).

Tabela 14 — Resumo dos testes T-01 a T-05

ID	Entradas	Saída observada	Resultado
T-01	endereço com A5A4 = 00, 01, 10, 11 (000000, 010000, 100000, 110000)	cs_rom, cs_ram, cs_reg, cs_io = 1 respectivamente, com os demais em 0. dado_saida = 0x11, 0x55, 0x00, 0x00	Aprovado
T-02	0x55 gravado (erro = 0); clock do Arranjo_Paridade desconectado; RAM regravada com 0x54 (bit 0 invertido)	Paridade guardada = 1 (de 0x55); recalculada = 0; erro_paridade = 1. Religando o clock, a paridade é regravada e erro = 0	Aprovado
T-03	cache zerada; endereço = 000000; leitura = 1; clk = 1	acerto = 0 (linha vazia). Após o pulso a linha guarda o rótulo 000 e o contador permanece em 0	Aprovado
T-04	depois do T-03; endereço = 000001	acerto = 1; contadorAcerto passa de 0000 para 0001	Aprovado
T-05	depois do T-03; endereço = 100000 (0x20)	acerto = 0; rotulo_linha passa de 000 para 100; contador inalterado	Aprovado

2.6 Traço de 16 acessos, planilha e indicadores

O traço obrigatório (16 leituras) foi classificado na planilha **parte1_planilha.xlsx**. As colunas endereço_bin, rotulo, indice, deslocamento, resultado e tipo_falta são obtidas por fórmula a partir de endereço_hex, e as colunas L0..L3 guardam o rótulo residente em cada linha após cada acesso. O traço foi também executado no circuito, lendo-se acerto com clk = 0 antes de cada pulso, e os 16 valores coincidiram com a planilha (Tabela 15, Figura 18). Como verificação adicional, a cache foi reescrita em Python de forma independente, e a simulação reproduziu a planilha nos 20 acessos (traço e teste de conflito).

Tabela 15 — Traço completo: acerto esperado (planilha) e observado (circuito)

Acesso	Endereço	Esperado	Observado	Resultado
1	0x00	0	0	FALTA
2	0x01	1	1	ACERTO

Acesso	Endereço	Esperado	Observado	Resultado
3	0x02	0	0	FALTA
4	0x03	1	1	ACERTO
5	0x00	1	1	ACERTO
6	0x20	0	0	FALTA
7	0x21	1	1	ACERTO
8	0x00	0	0	FALTA
9	0x04	0	0	FALTA
10	0x05	1	1	ACERTO
11	0x06	0	0	FALTA
12	0x07	1	1	ACERTO
13	0x22	0	0	FALTA
14	0x23	1	1	ACERTO
15	0x02	0	0	FALTA
16	0x03	1	1	ACERTO

Figura 18 — Traço completo: contador final em 8 e tabela de conferência

Indicadores do traço principal (Tabela 16), com $AMAT = \text{tempo de acerto} + (\text{taxa de faltas} \times \text{penalidade de falta})$:

Tabela 16 — Indicadores do traço principal

Indicador	Valor	Expressão
Acessos	16	—
Acertos	8	—
Faltas	8	$16 - 8$
Faltas compulsórias	4	linha vazia (acessos 1, 3, 9, 11)
Faltas por conflito	4	linha ocupada por outro rótulo (acessos 6, 8, 13, 15)
Taxa de acertos	50%	$8 / 16$
Taxa de faltas	50%	$8 / 16$
AMAT	5 ciclos	$1 + 0,5 \times 8$

2.7 Teste de conflito com o detector de primos

O Componente 17 é um detector de número primo de 4 bits, construído como soma de produtos com quatro portas AND (com NOTs nas entradas) e uma OR (Figura 19). A Tabela 17 é a sua tabela verdade; só são primos 2, 3, 5, 7, 11 e 13.

Tabela 17 — Tabela verdade do detector de primos

Entrada	Decimal	primo
0000	0	0
0001	1	0

Entrada	Decimal	primo
0010	2	1
0011	3	1
0100	4	0
0101	5	1
0110	6	0
0111	7	1
1000	8	0
1001	9	0
1010	10	0
1011	11	1
1100	12	0
1101	13	1
1110	14	0
1111	15	0

No circuito *teste_conflito*, um contador de 3 bits percorre $v = 0$ a 7. Para cada v primo, a leitura é habilitada e o endereço é v deslocado três posições à esquerda (v concatenado com 000). O endereço passa pelo *calc_endereco* (Componente 08): o valor de 6 bits é estendido com zeros para 8 bits (base), somado a deslocamento = 0x00, e os 6 bits baixos da soma (endereço_cache) vão ao cachetop (Figura 20 e Figura 21). Os pinos soma e cout ficam sem uso.

Interpretação adotada: o enunciado descreve o Componente 08 apenas como "cálculo do endereço a ser apresentado à cache". Os significados de base e deslocamento acima são escolha do grupo.

Tabela 18 — Pinos do calc_endereco

Pino	Bits	Dir.	Função
base	8	entrada	Endereço-base (v deslocado, estendido a 8 bits)
deslocamento	8	entrada	Constante 0x00
soma	8	saída	base + deslocamento (não usada)
cout	1	saída	Vai-um (não usado)
endereco_cache	6	saída	6 bits baixos da soma, enviados ao cachetop

Tabela 19 — Resultado do teste de conflito (cache invalidada no início)

v	Endereço	Binário	Rótulo	Índice	Resultado	rotulo_linha antes
2	0x10	010000	010	00	FALTA (compulsória)	000
3	0x18	011000	011	00	FALTA (conflito)	010
5	0x28	101000	101	00	FALTA (conflito)	011
7	0x38	111000	111	00	FALTA (conflito)	101

Nos quatro acessos, acerto = 0 e contadorAcerto terminou em 0000; a linha 0 passou pelos rótulos 000 → 010 → 011 → 101 → 111 (Figura 22). Indicadores do teste: 4 acessos, 0 acertos, 4 faltas (1 compulsória e 3 por conflito), taxa de acertos 0% e $AMAT = 1 + 1 \times 8 = 9$ ciclos.

Por que a regra produz este resultado. Deslocar v três posições à esquerda coloca v em A5A4A3 e deixa A2A1A0 = 000. Como o índice é A2A1, ele vale sempre 00: os quatro endereços caem na linha 0. Os rótulos A5A4A3 (010, 011, 101, 111) são diferentes, então cada acesso substitui o bloco anterior, e nenhum acerto acontece mesmo com três linhas ociosas.

Com oito linhas. O índice passaria a ter 3 bits (A3A2A1) e o rótulo 2 bits (A5A4). Os endereços ficariam como na Tabela 20. O 0x10 iria para outra linha, e o conflito diminuiria, mas não desapareceria: A2A1 continua 00, apenas A3 varia, e três dos quatro valores de v têm o bit menos significativo igual a 1. Nesse traço ainda não haveria acertos, porque nenhum endereço é lido duas vezes.

Tabela 20 — Os mesmos endereços com uma cache de 8 linhas

Endereço	Binário	Índice (A3A2A1)	Rótulo (A5A4)
0x10	010000	000	01
0x18	011000	100	01
0x28	101000	100	10
0x38	111000	100	11

2.8 Observações e limitações

- O Arranjo_Paridade usa o memoriaRAM16by de 8 bits e emprega só o bit 0; o enunciado pede um arranjo de 16 posições de 1 bit.
- No T-02, o bit invertido foi simulado desconectando o clock do Arranjo_Paridade e regravando a RAM com 0x54, de modo que a paridade guardada permaneceu a de 0x55.
- Os sinais we_ram e we_reg são pinos independentes, não derivados de cs_ram e cs_reg.
- A região de E/S (0x30–0x3F) não tem dispositivo; o multiplexador retorna 0x00.
- O banco de registradores tem uma única porta de leitura.
- A coluna tipo_falta fica vazia nos acertos. Uma falta é "compulsória" quando a linha estava vazia e "de conflito" quando estava ocupada por outro rótulo, como na Seção 5.5 do enunciado.

3. Parte II — Unidade de controle

Para a otimização extrema do circuito físico, o projeto tratou os estados não utilizados da máquina de estados (5, 6 e 7, correspondentes a 101, 110 e 111) como Condições Indiferentes (*Don't Cares* ou X). A aplicação desta técnica através dos Mapas de Karnaugh

permitiu a simplificação drástica das equações booleanas de próximo estado e dos sinais de saída.

O sinal de decisão de desvio foi extraído diretamente do bit 2 do Opcode, sendo denominado de sinal B, onde B=0 representa instruções do tipo-R e B=1 representa a instrução beq.

Diagrama de Estados

O diagrama de estados descreve o comportamento sequencial da Unidade de Controle para as instruções Tipo-R (Opcode 000000) e BEQ (Opcode 000100). O estado inicial de Busca é o S0.

- **S0 (Busca - 000):** Lê a instrução da memória. Transita incondicionalmente para S1.
- **S1 (Decodificação - 001):** Analisa o Opcode (sinal B).
 - Se B = 0 (Tipo-R), transita para S2.
 - Se B = 1 (BEQ), transita para S4.
- **S2 (Execução Tipo-R - 010):** A ULA processa os registradores. Transita incondicionalmente para S3.
- **S3 (Conclusão Tipo-R - 011):** O resultado é gravado no registrador de destino. Retorna incondicionalmente para S0.
- **S4 (Conclusão BEQ - 100):** Calcula o endereço de desvio e atualiza o PC. Retorna incondicionalmente para S0.

Tabela de Transição de Estados

A máquina possui 5 estados úteis. Os estados 5 (101), 6 (110) e 7 (111) são impossíveis de alcançar em operação normal e foram tratados como Condições Indiferentes (Don't Cares - x) para otimização lógica. O bit de entrada \$B\$ representa o bit 2 do Opcode.

Tabela de transições de estados				
Estado Atual (Q2Q1Q0)	Nome do Estado	Entrada B (Opcode)	Próximo Estado (D2D1D0)	Justificativa da Transição
000	S0 (Busca)	x	001	De S0 vai sempre para S1.
001	S1 (Decodificação)	0(tipo-R)	010	De S1, se for tipo-R, vai para S2.
001	S1 (Decodificação)	1(beq)	100	De S1, se for beq, vai para S4.
010	S2 (Execução tipo-R)	x	011	De S2 vai sempre para S3.
011	S3 (Conclusão tipo-R)	x	000	De S3 regressa sempre a S0.
100	S4 (Conclusão beq)	x	000	De S4 regressa sempre a S0.
101	(Não utilizado)	x	xxx	Estado inválido (Condição Indiferente).
110	(Não utilizado)	x	xxx	Estado inválido (Condição Indiferente).
111	(Não utilizado)	x	xxx	Estado inválido (Condição Indiferente).

Mapas de Karnaugh e Agrupamentos

Os mapas exploram as condições indiferentes (X) nas posições 101, 110 e 111.

Próximo Estado D1:

- Agrupamento 1 (Células 001 e 101): Resulta em $\text{Not}Q1 * Q0 * B$
- Agrupamento 2 (Células 010 e 110): Resulta em $Q1 * Q0$

Próximo Estado D2:

- Agrupamento 1 (Células 001 e 101 com B=1): Resulta em $\text{Not}Q1 * Q0 * B$

Sinal de Controle RegWr:

- Célula 011 tem valor 1. Agrupando com o X da célula 111, elimina-se a dependência de Q2. Resultado: $Q1 * Q0$

Equações Booleanas Finais (Soma de Produtos)

Através da minimização por Mapas de Karnaugh, obtiveram-se as seguintes equações matemáticas para a alimentação dos Flip-Flops D e para as portas lógicas da unidade cabeada:

Lógica de Próximo Estado:

$$\begin{aligned} D0 &= \text{NOT_Q2} * \text{NOT_Q1} * \text{NOT_Q0} \\ D1 &= (\text{NOT_Q1} * Q0 * B) + (Q1 * \text{NOT_Q0}) \\ D2 &= \text{NOT_Q1} * Q0 * B \end{aligned}$$

Lógica de Saída (Sinais de Controle selecionados):

$$\begin{aligned} \text{PCWr} &= \text{MemRd} = \text{IRWr} = \text{SrcB_0} = \text{NOT_Q1} * \text{NOT_Q0} \text{ (Condicionado ao S0)} \\ \text{RegWr} &= Q_ * Q_0 \\ \text{SrcA} &= Q1 * \text{NOT_Q0} \\ \text{PCWrC} &= \text{PCSrc} = \text{ALUOp_0} = Q_2 \\ \text{Circuito no Logisim e Contagem de Portas} \end{aligned}$$

O projeto foi integralmente implementado no Logisim-Evolution utilizando exclusivamente Flip-Flops D e portas lógicas discretas, respeitando o limite do projeto. A exibição do estado atual ciclo a ciclo foi consolidada na saída através de um Display Hexadecimal acoplado aos sinais de estado, substituindo eficientemente o decodificador externo de 7 segmentos.

Contagem Total de Recursos Físicos:

- 3 Registradores de 1 bit (Flip-Flops D)
- 9 Portas Lógicas no circuito combinacional, sendo:
 - 1 Porta NOT (Para inversão do sinal B)
 - 6 Portas AND (Para os produtos booleanos)
 - 2 Portas OR (Para a soma de produtos e sinais compartilhados)

O Microprograma (Parte 2)

Na implementação da Unidade de Controle Microprogramada, a lógica combinacional foi totalmente substituída por um Microcontador (PC) e uma memória ROM de 8x8 bits. Os sinais de controlo foram agregados numa microinstrução de 8 bits, dividida nos campos semânticos ULAOp (bits 7-6), Fonte (bits 5-4), Ação (bits 3-2) e Seq (bits 1-0).

Estado (Endereço)	ULAOp	Fonte	Ação	Seq	Binário	Hexadecimal
S0 (Busca)	00	00	01	00	00000100	4
S1 (Decodificação)	00	10	00	10	00100010	22
S2 (Execução Tipo-R)	10	01	00	00	10010000	90
S3 (Conclusão Tipo-R)	00	11	10	01	00111001	39
S4 (Conclusão BEQ)	01	01	11	01	01011101	5D

4. Declaração de uso de ferramentas de IA generativa

Ferramenta: Claude (Anthropic), usada pela interface web, em modo de professor particular de laboratório. Etapas e finalidade:

- Orientação passo a passo no Logisim-Evolution e conferência dos prints das telas do simulador durante os testes T-01 a T-05, o teste da Seção 5.6, o teste do banco de registradores e a ligação do calc_endereco.
- Preenchimento da planilha parte1_planilha.xlsx a partir do modelo fornecido em aula (extensão das fórmulas e do bloco Resumo) e verificação independente por simulação em Python.
- Geração do rascunho deste relatório, revisado pelo grupo.

A construção dos circuitos, a execução dos testes e a captura das telas foram feitas pelos integrantes.

5. Conclusão

O módulo de memória responde corretamente em cada região (T-01), o banco de registradores grava e lê os dados esperados, o erro de paridade é detectado (T-02) e a cache classifica corretamente faltas compulsórias, acertos no mesmo bloco e faltas por conflito (T-03 a T-05). O traço de 16 acessos resultou em 8 acertos e 8 faltas, com taxa de acertos de 50% e AMAT de 5 ciclos. O teste da Seção 5.6 mostrou o principal defeito do mapeamento direto: quatro endereços disputam uma mesma linha, e a taxa de acertos é 0%.

A abordagem cabeada é imperativa em arquiteturas RISC de alto desempenho moderno, onde o custo do silício, a dissipação térmica e a latência de propagação do sinal (portas lógicas são mais rápidas que a leitura de uma ROM) devem ser minimizados. A abordagem microprogramada é fundamental em arquiteturas CISC (como a linha x86), onde a complexidade e quantidade massiva das instruções exigem abstração, flexibilidade extrema e capacidade de atualização de firmware após o fabrico do processador.

Referências

UNIVERSIDADE FEDERAL DE RORAIMA. Laboratório de Circuitos — Projeto Integrador (versão introdutória). Prof. Dr. Herbert Oliveira Rocha, 2026.2.

LOGISIM-EVOLUTION. Documentação do simulador. Disponível em: <https://github.com/logisim-evolution/logisim-evolution>.